GIT & GITHUB PRACTICALS

Complete Beginner's Procedure — Practicals 3, 4, 5 and 6

This guide tells you exactly what to click, what to type, and where to type it.

Before you begin

• Use a Windows computer connected to the internet.

• For Practicals 3 and 4, use Git Bash. For Practicals 5 and 6, use GitHub Desktop.

• Commands go into the black Git Bash window—not into a Python file, Notepad, or your browser.

• Type one command at a time, then press Enter. Wait for it to finish before typing the next command.

• Replace example usernames and email addresses with your own. Do not share your password or access token.

• GitHub button names can vary slightly. If a step looks different, pause and check before proceeding.

How to use this PDF

1. Open this PDF on your computer so you can read it while working.

2. Start with Practical 3 and follow each step in order.

3. When a command is shown in a grey box, type only the command—not any explanation around it.

4. If you see an error, stop and read the troubleshooting section at the end.

Note: The procedures are based on the Practical 3–6 instructions provided for this task. Some GitHub screens can look different depending on account and repository settings.

PRACTICAL 3 — Git Bash: Branches, Pulling, Merge Conflict and Pull Request

Aim: Create a Git repository, upload it to GitHub, create two branches, pull changes between branches, demonstrate a merge conflict, and create a pull request.

Part A — Create the project folder

1. Go to your Windows Desktop.

2. Right-click an empty area → click **New** → click **Folder**.

3. Type **DataScienceProject** and press Enter.

4. Double-click the new folder to open it.

5. Right-click an empty area inside the folder.

6. Click **Open Git Bash here**. A black terminal window should appear.

If you do not see "Open Git Bash here": Git may not be installed, or the menu option may be unavailable. Stop and tell your instructor/helper rather than typing commands in the wrong window.

Part B — Configure your name and email (first time only)

In the black Git Bash window, type the first command and press Enter. Then type the second command and press Enter. Replace the examples with your details.

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

Part C — Start Git in this folder

Make sure Git Bash is open in the DataScienceProject folder. Type:

```
git init
```

Git starts tracking this folder. You may see a message that an empty repository was initialized.

Part D — Create an empty repository on GitHub

1. Open your web browser and go to **https://github.com**.

2. Sign in to your GitHub account.

3. Click the **+** button near the top-right of the page.

4. Click **New repository**.

5. In Repository name, type **DataScienceProject**.

6. Choose Public or Private according to your preference.

7. Leave **Add a README file** unchecked.

8. Click **Create repository**.

9. On the next page, find the HTTPS repository address and copy it. It looks like https://github.com/YOUR-USERNAME/DataScienceProject.git.

Part E — Connect your folder to GitHub and push main

Return to the black Git Bash window. Replace the sample URL in the first line with the exact URL you copied. Keep the whole first command on one line.

```
git remote add origin https://github.com/YOUR-USERNAME/DataScienceProject.git
```

Now type each command separately:

```
git branch -M main
git commit --allow-empty -m "Initial commit"
git push -u origin main
```

If a browser opens for sign-in, complete the sign-in there. After the push succeeds, your main branch is on GitHub.

Part F — Create branch GIT-EXAM-1

```
git checkout -b GIT-EXAM-1
```

This creates the branch and switches to it. The terminal usually confirms that you switched to the new branch.

Part G — Create program1.py (print squares)

1. In Git Bash, type the command below and press Enter.

2. If Notepad asks whether to create the file, click **Yes**.

3. Copy the Python code below into Notepad.

4. In Notepad, click **File** → **Save**, then close Notepad.

```
notepad program1.py
```

```
for number in range(1, 11):
    print(number, "square =", number * number)
```

Part H — Create program2.py (palindrome checker)

A palindrome is a word that reads the same forward and backward, such as madam.

```
notepad program2.py
```

Paste this code into Notepad, then click File → Save and close Notepad:

```
text = input("Enter a word: ")

if text == text[::-1]:
    print("It is a palindrome")
else:
    print("It is not a palindrome")
```

Part I — Save and upload the two programs

In Git Bash, type these commands one at a time:

```
git add program1.py program2.py
git commit -m "Add Python programs on GIT-EXAM-1"
git push -u origin GIT-EXAM-1
```

To check: open your repository on GitHub, click the branch dropdown, choose GIT-EXAM-1, and check that program1.py and program2.py appear.

Part J — Create GIT-EXAM-2 and pull changes

```
git checkout -b GIT-EXAM-2
git pull origin GIT-EXAM-1
```

The first command creates the second branch. The second command brings changes from GIT-EXAM-1 into your current branch.

Part K — Change program1.py on GIT-EXAM-2 to print cubes

```
notepad program1.py
```

Replace the whole file with this code. Save and close Notepad.

```
for number in range(1, 11):
    print(number, "cube =", number * number * number)
```

```
git add program1.py
git commit -m "Change squares to cubes on GIT-EXAM-2"
git push -u origin GIT-EXAM-2
```

PRACTICAL 3 — continued

Part L — Change the same file on GIT-EXAM-1 to print doubles

This different change is needed to demonstrate a merge conflict.

```
git checkout GIT-EXAM-1
notepad program1.py
```

Replace the whole file with the code below. Save and close Notepad.

```
for number in range(1, 11):
    print(number, "double =", number * 2)
```

```
git add program1.py
git commit -m "Change squares to doubles on GIT-EXAM-1"
git push
```

Part M — Check the branches on GitHub

1. Open your DataScienceProject repository on GitHub.

2. Click the branch dropdown (it may show main).

3. Select GIT-EXAM-1 and check program1.py.

4. Select GIT-EXAM-2 and check program1.py. The two branches have different versions.

Part N — Create a pull request

1. On your repository page, click the **Pull requests** tab.

2. Click **New pull request**.

3. Set **base** to GIT-EXAM-1.

4. Set **compare** to GIT-EXAM-2.

5. Click **Create pull request**.

6. Enter a title such as **Merge GIT-EXAM-2 into GIT-EXAM-1**.

7. Click **Create pull request** again.

GitHub may show that the branches cannot be merged automatically. This is expected because both branches changed the same file differently.

Part O — Resolve the merge conflict

Return to Git Bash and type:

```
git checkout GIT-EXAM-2
git pull origin GIT-EXAM-1
```

If Git reports a conflict, open the file:

```
notepad program1.py
```

You may see marker lines like these. Do not leave these marker lines in the final Python file:

```
<<<<<<< HEAD
(your version)
=======
(other branch version)
>>>>>>> GIT-EXAM-1
```

Delete all marker lines and choose the final code to keep. For example, use this squares version:

```
for number in range(1, 11):
    print(number, "square =", number * number)
```

Save and close Notepad. Then type:

```
git add program1.py
git commit -m "Resolve merge conflict in program1.py"
git push
```

Refresh the pull request on GitHub. The conflict should be resolved. A person with permission can merge the pull request.

Result: Created and pushed two branches, opened a pull request, demonstrated a merge conflict, and resolved it.

PRACTICAL 4 — Git Bash: Data Science Project and .gitignore

Aim: Create a repository, add Python programs, create a branch, and use .gitignore to prevent a folder from being uploaded.

Part A — Create folder and open Git Bash

1. On the Desktop, right-click → New → Folder.

2. Name it DataScienceProject and press Enter.

3. Open the folder, right-click inside it, and choose Open Git Bash here.

Part B — Initialize Git

If Git name and email are not configured, use the two commands below with your own details. Then initialize the repository.

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
git init
```

Part C — Create the Python files

Create the first program:

```
notepad program1.py
```

```
for number in range(1, 11):
    print(number, "square =", number * number)
```

Save and close Notepad. Now create the second program:

```
notepad program2.py
```

```
text = input("Enter a word: ")

if text == text[::-1]:
    print("It is a palindrome")
else:
    print("It is not a palindrome")
```

Save and close Notepad.

Part D — Run the programs

Type these commands in Git Bash, one at a time:

```
python program1.py
python program2.py
```

If the command python is not recognized, try py program1.py and py program2.py. When program2 asks for a word, type madam and press Enter.

Part E — Create the GitHub repository

1. Open https://github.com and sign in.

2. Click + → New repository.

3. Name the repository DataScienceProject.

4. Choose Public or Private. Leave Add a README file unchecked.

5. Click Create repository and copy the HTTPS URL.

Part F — Connect and push your files

In Git Bash, replace the sample URL with your own repository URL. If you already added a remote named origin in this same folder, do not repeat the first command.

```
git remote add origin https://github.com/YOUR-USERNAME/DataScienceProject.git
git add program1.py program2.py
git commit -m "Add Python programs"
git branch -M main
git push -u origin main
```

Part G — Create branch GIT-EXAM-1

```
git checkout -b GIT-EXAM-1
```

```
git push -u origin GIT-EXAM-1
```

If Git says there is nothing to commit, that is okay: the files may already be committed. The push command publishes the branch.

Part H — Create the IGNORE folder and text files

Type these commands in Git Bash:

```
mkdir IGNORE
touch IGNORE/test1.txt
touch IGNORE/test2.txt
```

These commands create a folder named IGNORE and two empty text files inside it.

Part I — Create .gitignore

```
notepad .gitignore
```

If asked to create the file, click Yes. Type exactly this one line into Notepad:

```
IGNORE/
```

Click File → Save and close Notepad. Make sure the file is named .gitignore, not .gitignore.txt. To show extensions in File Explorer: View → Show → File name extensions.

Part J — Check and upload .gitignore

```
git status
```

You should see .gitignore as a new or changed file. You should not see IGNORE/test1.txt or IGNORE/test2.txt as files to upload.

```
git add .gitignore
git commit -m "Ignore IGNORE directory"
git push
```

Part K — Verify on GitHub

1. Open your repository on GitHub.

2. Use the branch dropdown to select GIT-EXAM-1.

3. Check that program1.py, program2.py and .gitignore appear.

4. Check that the IGNORE folder does not appear.

Result: Added two Python programs and used .gitignore to prevent the IGNORE folder from being uploaded as new files.

PRACTICAL 5 — GitHub Desktop: Data Science Project and .gitignore

Aim: Use GitHub Desktop to publish a repository, create a branch, add Python programs and ignore a folder.

Part A — Create a project folder

1. Go to your Desktop.

2. Right-click an empty area → New → Folder.

3. Name the folder DataScienceProject and press Enter.

Part B — Create the repository in GitHub Desktop

1. Open GitHub Desktop from the Windows Start menu.

2. Sign in to GitHub if it asks.

3. At the top, click File → Add Local Repository.

4. Click Choose and select Desktop → DataScienceProject.

5. Click Select Folder.

6. If GitHub Desktop says this is not a Git repository, click the option to create a repository.

7. In the create window, set Name to DataScienceProject and check the Local Path points to your Desktop.

8. Click Create Repository.

Part C — Publish the repository

1. In GitHub Desktop, click Publish repository near the top.

2. Check the repository name is DataScienceProject.

3. If you want it private, leave Keep this code private checked.

4. Click Publish Repository and wait for the upload to finish.

Part D — Create branch GIT-EXAM-1

1. At the top, click Current Branch.

2. Click New Branch.

3. Type GIT-EXAM-1.

4. Click Create Branch.

5. Check the current branch shown at the top is GIT-EXAM-1.

Part E — Create program1.py

1. In GitHub Desktop, click Repository → Show in Explorer.

2. A File Explorer window opens for DataScienceProject.

3. Right-click an empty area → New → Text Document.

4. Name it program1.py. If Windows asks whether to change the extension, click Yes.

5. Right-click program1.py → Open with → Notepad.

6. Paste the code below, then File → Save and close Notepad.

```
for number in range(1, 11):
    print(number, "square =", number * number)
```

Part F — Create program2.py

In the same project folder, create another text file named program2.py. Open it in Notepad and paste this code:

```
text = input("Enter a word: ")

if text == text[::-1]:
    print("It is a palindrome")
```

```
else:
    print("It is not a palindrome")
```

Save and close Notepad. If Windows hides extensions, check that the names are not program1.py.txt or program2.py.txt.

Part G — Run the programs (optional check)

1. Open Command Prompt or PowerShell from the Windows Start menu.

2. Type cd followed by one space. Do not press Enter yet.

3. Drag the DataScienceProject folder from the Desktop into the terminal window.

4. Press Enter.

5. Type py program1.py and press Enter.

6. Then type py program2.py and press Enter. Type madam when it asks for a word.

Part H — Commit and push the programs

1. Return to GitHub Desktop.

2. Under Changes, you should see program1.py and program2.py.

3. At the bottom-left, find the Summary box and type Add two Python programs.

4. Click Commit to GIT-EXAM-1.

5. Click Push origin near the top and wait for it to finish.

Part I — Create IGNORE and its files

1. Open the DataScienceProject folder again.

2. Right-click empty space → New → Folder. Name it IGNORE.

3. Open IGNORE.

4. Right-click empty space → New → Text Document. Name it test1.txt.

5. Repeat to create test2.txt. If asked about extensions, confirm .txt is correct.

6. You can open each file in Notepad, type a short line, save and close.

Part J — Create .gitignore

1. Go back to the main DataScienceProject folder, not inside IGNORE.

2. Right-click empty space → New → Text Document.

3. Name the file .gitignore. If Windows asks about the extension, click Yes.

4. Open .gitignore with Notepad and type exactly:

```
IGNORE/
```

Click File → Save and close Notepad. The name must be .gitignore, not .gitignore.txt.

Part K — Commit and push .gitignore

1. Open GitHub Desktop and check that the current branch is GIT-EXAM-1.

2. Under Changes, .gitignore should appear. The IGNORE files should not appear as new files.

3. In Summary, type Add gitignore to exclude IGNORE folder.

4. Click Commit to GIT-EXAM-1.

5. Click Push origin.

Part L — Verify on GitHub

1. In GitHub Desktop, click Repository → View on GitHub.

2. On the website, select the GIT-EXAM-1 branch.

3. Check that program1.py, program2.py and .gitignore appear.

4. Check that IGNORE is not shown.

Result: Published the project using GitHub Desktop and used .gitignore to exclude the IGNORE folder.

PRACTICAL 6 — GitHub Desktop: Clone, Fork, Branch and Pull Request

Aim: Clone and fork an AI project repository, create a feature branch named GIT-EXAM, make a change, submit a pull request, and understand merge conflicts.

Repository in the practical: https://github.com/KalyanM45/AI-Project-Gallery

Part A — Clone the original repository

1. Open GitHub Desktop.

2. At the top, click File → Clone Repository.

3. In the window, select the URL tab.

4. In Repository URL, paste: https://github.com/KalyanM45/AI-Project-Gallery.git

5. Under Local Path, click Choose.

6. In the folder picker, click Desktop on the left.

7. Click Select Folder.

8. Back in GitHub Desktop, click Clone and wait for it to finish.

Part B — Try creating the branch and understand permissions

1. In GitHub Desktop, find Current Branch near the top.

2. Click Current Branch → New Branch.

3. Type GIT-EXAM and click Create Branch.

Creating a local branch is usually allowed. You may not be allowed to push to the original owner's repository. The next steps use a fork so you have your own copy.

Part C — Fork the repository on GitHub

1. Open your web browser and go to https://github.com/KalyanM45/AI-Project-Gallery.

2. Sign in to GitHub.

3. Click Fork near the top-right of the repository page.

4. Under Owner, select your own GitHub account.

5. Click Create fork and wait for your copy to open.

Part D — Clone your own fork

1. On your fork page, click the green Code button.

2. Copy the HTTPS URL. It should contain your own username, like https://github.com/YOUR-USERNAME/AI-Project-Gallery.git.

3. Return to GitHub Desktop and click File → Clone Repository.

4. Select URL and paste the URL for your fork—not the original repository URL.

5. Under Local Path, click Choose and select Desktop.

6. Click Select Folder, then click Clone.

Part E — Create the GIT-EXAM branch on your fork

1. At the top of GitHub Desktop, click Current Branch.

2. Click New Branch.

3. Type GIT-EXAM.

4. Click Create Branch.

5. Check that the top of GitHub Desktop now shows GIT-EXAM.

Part F — Edit README.md

1. In GitHub Desktop, click Repository → Show in Explorer.

2. Find README.md in the project folder.

3. Right-click README.md and choose Open with → Notepad. If that option is missing, open Notepad first, then File → Open and select README.md.

4. Go to the bottom of the file and add this line:

```
This change is for the GIT-EXAM practical.
```

Save the file using File → Save, then close Notepad.

Part G — Commit and push the change

1. Return to GitHub Desktop. README.md should appear under Changes.

2. In the Summary box at the bottom-left, type Update README for GIT-EXAM.

3. Click Commit to GIT-EXAM.

4. Click Push origin near the top and wait for it to finish.

Part H — Open a pull request

1. Open your fork on GitHub in the browser.

2. If you see Compare & pull request, click it. Otherwise, click Pull requests → New pull request.

3. Set the base repository to KalyanM45/AI-Project-Gallery (the original repository).

4. Set the base branch to main or the default branch shown by GitHub.

5. Set the compare repository to your fork (the one under your username).

6. Set the compare branch to GIT-EXAM.

7. Enter a title, such as Changes for GIT-EXAM practical.

8. Click Create pull request.

A pull request asks the original owner to review your changes. It does not mean the changes have already been accepted.

Part I — If the reviewer asks for changes

1. Read the reviewer's comments on the pull request page.

2. In GitHub Desktop, click Repository → Show in Explorer.

3. Open the requested file, make the change, save and close it.

4. Return to GitHub Desktop and type Apply review feedback in Summary.

5. Click Commit to GIT-EXAM.

6. Click Push origin. The new commit will update the same pull request.

Part J — If a merge conflict happens

1. Open the conflicting file in a text editor.

2. Find marker lines such as <<<<<<<, ======= and >>>>>>>.

3. Choose the content you want to keep, or combine the two versions.

4. Delete the marker lines and make sure the file contains valid content.

5. Save the file.

6. Return to GitHub Desktop, enter a commit message and commit the resolved file.

7. Click Push origin and refresh the pull request.

Result: Cloned and forked the repository, created the GIT-EXAM branch, committed a README change, pushed it, and opened a pull request.

QUICK VIVA REVISION AND TROUBLESHOOTING

Important words in easy English

• **Git:** A tool that records changes to files.

• **GitHub:** A website where Git repositories can be stored and shared.

• **Repository:** A project folder tracked by Git.

• **Branch:** A separate line of work in a project.

• **git init:** Starts a Git repository in the current folder.

• **git add:** Selects changes to include in the next commit.

• **Commit:** A saved point in the project's history.

• **Push:** Uploads local commits to GitHub.

• **Pull:** Brings remote changes into the current branch.

• **Clone:** Downloads a repository to your computer.

• **Fork:** Makes your own GitHub copy of another person's repository.

• **Pull request:** Asks for changes to be reviewed and possibly merged.

• **Merge conflict:** Git cannot automatically combine different changes.

• **.gitignore:** Tells Git to ignore specified untracked files or folders.

• **Palindrome:** A word that reads the same forward and backward, e.g. madam.

Common errors and what to do

• **“git is not recognized”:** Git may not be installed or Git Bash is not open. Open Git Bash from the Start menu or install Git if needed.

• **“Author identity unknown”:** Set your Git name and email using git config commands shown earlier.

• **“remote origin already exists”:** Do not repeat git remote add origin. The folder may already be connected to GitHub.

• **“nothing to commit”:** There may be no new changes to save. Check the file and run git status.

• **Python command fails:** Try py program1.py instead of python program1.py. Python must be installed to run the programs.

• **GitHub says permission denied when pushing:** Check that you are signed in. For another person's repository, fork it and push to your fork.

• **.gitignore.txt appears:** Rename it to exactly .gitignore. Enable file name extensions in File Explorer to check.

• **Conflict markers remain in the file:** Open the file, choose the correct content, delete marker lines, save, then add, commit and push.

Before uploading this PDF to GitHub

1. Save this PDF to your computer using the download link provided in the chat.

2. Open https://github.com in your browser and sign in.

3. Open the repository where you want to keep the practical instructions.

4. Click Add file → Upload files.

5. Click choose your files or drag the PDF from your Downloads folder into the upload area.

6. Scroll down to the Commit changes section.

7. Keep the suggested commit message or type Add beginner guide for Git practicals 3 to 6.

8. Click Commit changes and wait for GitHub to finish uploading.

9. Open the repository page and click the PDF filename to view it.

Suggested PDF filename: Complete_Beginner_Git_Practicals_3_to_6.pdf